

OpenClaw Inference Invoice

Invoice Number: INV-2026-04-002
Billing Period: April 1–9, 2026 (9 days)
Generated: April 8, 2026 22:07 UTC
Provider: OpenClaw (local RTX 3090 inference)
Email: Jeremy.ingalls@gmail.com

Token Usage Summary

Metric	Total
Input Tokens	1,002,126
Output Tokens	66,706
Total Tokens	1,068,832

Breakdown by day: - April 5: 0 tokens (baseline reset day) - April 6: 634,273 input, 8,420 output
- April 7: 521,273 input, 48,288 output
- April 8: -153,420 input (reset occurred), 11,659 output - April 9: 0 tokens (new baseline)

Note: April 8 shows negative input tokens due to the daily reset occurring at 12:01 UTC. The reset subtracts the previous day’s tokens from the current day’s baseline.

Cost Breakdown

Local Costs (RTX 3090)

Component	Rate	Amount	Cost
Input tokens	\$0.04/M	1,002,126	\$0.0401
Output tokens	\$0.15/M	66,706	\$0.0100
Total Local			\$0.0501

Cloud Equivalent (Claude Opus 4.6)

Component	Rate	Amount	Cost
Input tokens	\$15.00/M	1,002,126	\$15.03
Output tokens	\$75.00/M	66,706	\$5.00
Total Cloud			\$20.03

Savings

Metric	Value
Total Savings	\$19.98
Savings Percentage	99.75%

Infrastructure Details

Hardware: NVIDIA RTX 3090 (24GB VRAM)

Acquisition Cost: ~\$900

Electricity Rate: \$0.155/kWh (Louisiana residential)

GPU Power Draw: ~300W sustained inference

Depreciation: 20% annually (\$180/year)

Software Stack: - llama.cpp (llama-server) - Qwen 3.5 35B GGUF model - Metrics scraping via Prometheus endpoint - PDF generation via pandoc + xelatex

Performance Metrics

Average Throughput: - Prefill (input): ~1,642 tokens/second - Decode (output): ~73 tokens/second

Total Processing Time: - Input: ~612 seconds (10.2 minutes) - Output: ~915 seconds (15.3 minutes)

Notes

- Invoices generated on-demand and automatically on the 1st of each month
- Daily metrics scraped at 12:00 UTC
- Baseline reset occurs after daily cost report
- All costs based on measured benchmarks from production server
- Invoice includes full month's data (April 1-9)

Total Due: \$0.0501 USD

Paid via electricity + depreciation

— *Sam*